

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME:** Database Management Systems

**COURSE CODE:** CSA05

---

### COURSE OUTCOME COVERED

**CO3:** *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*  
(BL2, BL3)

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

---

| QUESTIONS |                                                                                                                                                         |                  | Total Marks: 50 |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|-----------------|
| Q.No      | Question                                                                                                                                                | Bloom's Level    | Marks           |
| 1         | Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF. | BL2 – Understand | 5               |
| 2         | A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.                   | BL3 – Apply      | 5               |
| 3         | Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.                  | BL3 – Apply      | 5               |
| 4         | Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.                               | BL3 – Apply      | 5               |
| 5         | Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.                      | BL2 – Understand | 5               |
| 6         | Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.                                      | BL3 – Apply      | 5               |

|           |                                                                                                                                        |                  |          |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------|------------------|----------|
| <b>7</b>  | Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.      | BL2 – Understand | <b>5</b> |
| <b>8</b>  | A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization. | BL3 – Apply      | <b>5</b> |
| <b>9</b>  | Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.         | BL3 – Apply      | <b>5</b> |
| <b>10</b> | Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.              | BL3 – Apply      | <b>5</b> |

---

***Code of Conduct:***

I CH. Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

***Signature of the student:***

***CH. Durga Prasad***

**1. Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.**

### **Given Relation**

**Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)**

Assume:

- Each OrderID identifies one order.
- A customer can place many orders.
- A product can appear in many orders.
- Qty represents the quantity of a product in an order.
- Price is the price of the product.

### **1. Functional Dependencies (FDs)**

The main functional dependencies are:

1. **OrderID  $\rightarrow$  CustID**
  - An order belongs to one customer.
2. **CustID  $\rightarrow$  CustName**
  - Each customer ID determines the customer name.
3. **ProdID  $\rightarrow$  ProdName, Price**
  - Each product ID determines its product name and price.
4. **(OrderID, ProdID)  $\rightarrow$  Qty**
  - The quantity depends on the particular product within a particular order.

Therefore, the complete FD set can be written as:

**OrderID  $\rightarrow$  CustID**

**CustID  $\rightarrow$  CustName**

**ProdID  $\rightarrow$  ProdName, Price**

**(OrderID, ProdID)  $\rightarrow$  Qty**

### **2. Candidate Key**

Since an order can contain multiple products, OrderID alone is not sufficient.

The combination:

**(OrderID, ProdID)**

uniquely identifies each order-product entry.

So,

**Candidate Key = {OrderID, ProdID}**

### 3. Problems in the Original Relation

The original relation contains redundancy:

| OrderID | CustID | CustName | ProdID | ProdName | Qty | Price |
|---------|--------|----------|--------|----------|-----|-------|
| O101    | C01    | Durga    | P01    | Laptop   | 1   | 50000 |
| O101    | C01    | Durga    | P02    | Mouse    | 2   | 1000  |
| O102    | C02    | Ravi     | P01    | Laptop   | 1   | 50000 |

For example, customer name **Durga** is repeated for every product in order O101, and product information is repeated across different orders.

This can cause:

- **Update anomaly:** Changing a product price requires multiple updates.
- **Insertion anomaly:** A new product cannot easily be stored without an order.
- **Deletion anomaly:** Deleting the last order for a product may remove the product information.

### 4. Conversion to 3NF

**Table 1: CUSTOMER**

customer(custid, custname)

- Primary Key: CustID
- FD: CustID  $\rightarrow$  CustName

This removes the transitive dependency:

**OrderID  $\rightarrow$  CustID  $\rightarrow$  CustName**

**Table 2: PRODUCT**

product(prodid, prodname, price)

- Primary Key: ProdID
- FD: ProdID  $\rightarrow$  ProdName, Price

**Table 3: ORDERS**

orders(orderid, custid)

- Primary Key: OrderID

- Foreign Key: CustID
- FD: OrderID → CustID

**Table 4: ORDER\_ITEM**

order\_item(orderid, prodid, qty)

- Composite Primary Key: (OrderID, ProdID)
- Foreign Keys: OrderID, ProdID
- FD: (OrderID, ProdID) → Qty

**SQL:**

The screenshot shows the OneCompiler MySQL interface. The SQL code defines four tables: CUSTOMER, PRODUCT, ORDERS, and ORDER\_ITEM. It includes primary keys, foreign keys, and data insertion statements. The output shows a table with 7 columns: OrderID, CustID, CustName, ProdID, ProdName, Qty, and Price. The data is as follows:

| OrderID | CustID | CustName | ProdID | ProdName | Qty | Price    |
|---------|--------|----------|--------|----------|-----|----------|
| 1001    | 101    | Durga    | 201    | Laptop   | 1   | 50000.00 |
| 1001    | 101    | Durga    | 202    | Mouse    | 2   | 1000.00  |
| 1002    | 102    | Ravi     | 201    | Laptop   | 1   | 50000.00 |
| 1002    | 102    | Ravi     | 203    | Keyboard | 1   | 2000.00  |
| 1003    | 103    | Priya    | 202    | Mouse    | 3   | 1000.00  |

## Result

The original Order relation is decomposed into CUSTOMER, PRODUCT, ORDERS, and ORDER\_ITEM, eliminating partial and transitive dependencies and producing a 3NF design with reduced redundancy and fewer insertion, deletion, and update anomalies.

## 2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF

Hospital Relation

Given:

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Assume the following functional dependencies:

1.  $PID \rightarrow PName, DocID, WardNo$
2.  $DocID \rightarrow DocName, Specialization$
3.  $WardNo \rightarrow WardName$

### 1. Identify Anomalies

The original table has significant redundancy.

| PID | PName | DocID | DocName   | Specialization | WardNo | WardName |
|-----|-------|-------|-----------|----------------|--------|----------|
| P01 | Durga | D01   | Dr. Ravi  | Cardiology     | W01    | General  |
| P02 | Priya | D01   | Dr. Ravi  | Cardiology     | W01    | General  |
| P03 | Arun  | D02   | Dr. Kumar | Neurology      | W02    | ICU      |

### Update Anomaly

If Dr. Ravi changes specialization, every record containing D01 must be updated.

Insertion Anomaly

A new doctor cannot be added unless a patient is assigned to that doctor.

Deletion Anomaly

If the last patient assigned to a doctor is deleted, the doctor's information may also be lost.

Redundancy

DocName, Specialization, and WardName are repeatedly stored for multiple patients.

### 2. Candidate Key

From:

$PID \rightarrow PName, DocID, WardNo$

PID determines all other attributes.

Therefore:

Candidate Key = PID

### 3. BCNF Analysis

A relation is in BCNF if, for every non-trivial functional dependency:

$X \rightarrow Y$

X must be a superkey.

Consider:

$DocID \rightarrow DocName, Specialization$

DocID is not a superkey of the original relation because it does not determine PID.

Therefore, this violates BCNF.

**Similarly:**

$\text{WardNo} \rightarrow \text{WardName}$

WardNo is not a superkey, so this also violates BCNF.

**4. BCNF Decomposition****Table 1: PATIENT**

PATIENT(PID, PName, DocID, WardNo)

Primary Key: PID

FD:

$\text{PID} \rightarrow \text{PName}, \text{DocID}, \text{WardNo}$

PID is a superkey, so this relation satisfies BCNF.

**Table 2: DOCTOR**

DOCTOR(DocID, DocName, Specialization)

Primary Key: DocID

FD:

$\text{DocID} \rightarrow \text{DocName}, \text{Specialization}$

DocID is a superkey, so this relation satisfies BCNF.

**Table 3: WARD**

WARD(WardNo, WardName)

Primary Key: WardNo

FD:

$\text{WardNo} \rightarrow \text{WardName}$

WardNo is a superkey, so this relation satisfies BCNF.

**SQL:**

```

1 CREATE TABLE DOCTOR (
2   DocID INT PRIMARY KEY,
3   DocName VARCHAR(50) NOT NULL,
4   Specialization VARCHAR(50) NOT NULL
5 );
6
7 CREATE TABLE WARD (
8   WardNo INT PRIMARY KEY,
9   WardName VARCHAR(50) NOT NULL
10 );
11
12 CREATE TABLE PATIENT (
13   PID INT PRIMARY KEY,
14   PName VARCHAR(50) NOT NULL,
15   DocID INT,
16   WardNo INT,
17   FOREIGN KEY (DocID) REFERENCES DOCTOR(DocID),
18   FOREIGN KEY (WardNo) REFERENCES WARD(WardNo)
19 );
20
21 INSERT INTO DOCTOR VALUES
22 (101, 'Dr. Ravi', 'Cardiology'),
23 (102, 'Dr. Kumar', 'Neurology'),
24 (103, 'Dr. Priya', 'Orthopedics');
25
26 INSERT INTO WARD VALUES
27 (1, 'General Ward'),
28 (2, 'ICU'),
29 (3, 'Orthopedic Ward');
30
31 INSERT INTO PATIENT VALUES
32 (1, 'Durga', 101, 1),
33 (2, 'Priya', 101, 1),
34 (3, 'Arun', 102, 2),
35 (4, 'Rahul', 103, 3);
36
37 SELECT
38   P.PID,
39   P.PName,
40   D.DocID,
41   D.DocName,
42   D.Specialization,
43   W.WardNo,
44   W.WardName
45 FROM PATIENT P
46 JOIN DOCTOR D ON P.DocID = D.DocID
47 JOIN WARD W ON P.WardNo = W.WardNo;

```

Output:

| PID | PName | DocID | DocName   | Specialization | WardNo | WardName        |
|-----|-------|-------|-----------|----------------|--------|-----------------|
| 1   | Durga | 101   | Dr. Ravi  | Cardiology     | 1      | General Ward    |
| 2   | Priya | 101   | Dr. Ravi  | Cardiology     | 1      | General Ward    |
| 3   | Arun  | 102   | Dr. Kumar | Neurology      | 2      | ICU             |
| 4   | Rahul | 103   | Dr. Priya | Orthopedics    | 3      | Orthopedic Ward |

## Conclusion:

The original Patient relation violates **BCNF** because DocID and WardNo determine attributes without themselves being superkeys. Decomposing it into **PATIENT**, **DOCTOR**, and **WARD** removes redundancy and update, insertion, and deletion anomalies while producing a **BCNF design**.

## 3. Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition

### Case Study: University Enrollment

Given relation:

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

### 1. Functional Dependencies

Assume:

- $SID \rightarrow SName$ 
  - Student ID determines student name.
- $CourseID \rightarrow CourseName, Faculty$ 
  - Course ID determines course name and faculty.
- $(SID, CourseID) \rightarrow Grade$ 
  - A student's grade depends on both the student and the course.

Therefore:

$SID \rightarrow SName$

$CourseID \rightarrow CourseName, Faculty$

$(SID, CourseID) \rightarrow Grade$

## 2. Candidate Key

A student can enroll in multiple courses, and a course can have multiple students.

Therefore, the candidate key is:

$(SID, CourseID)$

## 3. Why It Violates 2NF

The relation is initially assumed to be in 1NF.

However, it violates 2NF because there are partial dependencies on parts of the composite key:

$SID \rightarrow SName$

Here, SName depends only on SID.

And:

$CourseID \rightarrow CourseName, Faculty$

Here, CourseName and Faculty depend only on CourseID.

But:

$(SID, CourseID) \rightarrow Grade$

Grade depends on the complete composite key, so this dependency is valid in 2NF.

## 4. 2NF Decomposition

STUDENT

STUDENT(SID, SName)

Primary Key: SID

$SID \rightarrow SName$

COURSE

COURSE(CourseID, CourseName, Faculty)

Primary Key: CourseID

$CourseID \rightarrow CourseName, Faculty$

ENROLLMENT

ENROLLMENT(SID, CourseID, Grade)

Composite Primary Key: (SID, CourseID)

$(SID, CourseID) \rightarrow Grade$

## SQL:

The screenshot shows the OneCompiler SQL editor with a MySQL database. The SQL code defines three tables: STUDENT, COURSE, and ENROLLMENT. The STUDENT table has columns SID (primary key) and SName. The COURSE table has columns CourseID (primary key), CourseName, and Faculty. The ENROLLMENT table has columns SID, CourseID (composite primary key), and Grade. It also includes foreign key constraints for STUDENT and COURSE. The code then inserts data into these tables and finally selects all records from the ENROLLMENT table.

```
1 CREATE TABLE STUDENT (
2   SID INT PRIMARY KEY,
3   SName VARCHAR(50) NOT NULL
4 );
5
6 CREATE TABLE COURSE (
7   CourseID INT PRIMARY KEY,
8   CourseName VARCHAR(100) NOT NULL,
9   Faculty VARCHAR(50) NOT NULL
10 );
11
12 CREATE TABLE ENROLLMENT (
13   SID INT,
14   CourseID INT,
15   Grade VARCHAR(5),
16   PRIMARY KEY (SID, CourseID),
17   FOREIGN KEY (SID) REFERENCES STUDENT(SID),
18   FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
19 );
20
21 INSERT INTO STUDENT VALUES
22 (1, 'Durga'),
23 (2, 'Ravi'),
24 (3, 'Priya'),
25 (4, 'Arun');
26
27 INSERT INTO COURSE VALUES
28 (101, 'Database Management Systems', 'Dr. Kumar'),
29 (102, 'Data Structures', 'Dr. Ravi'),
30 (103, 'Machine Learning', 'Dr. Priya');
31
32 INSERT INTO ENROLLMENT VALUES
33 (1, 101, 'A'),
34 (1, 102, 'B+'),
35 (2, 101, 'A'),
36 (2, 102, 'A'),
37 (3, 102, 'A'),
38 (4, 103, 'B+');
39
40 SELECT
41   S.SID,
42   S.SName,
```

The output of the SQL query is displayed in a table format:

| SID | SName | CourseID | CourseName                  | Faculty   | Grade |
|-----|-------|----------|-----------------------------|-----------|-------|
| 1   | Durga | 101      | Database Management Systems | Dr. Kumar | A     |
| 2   | Ravi  | 101      | Database Management Systems | Dr. Kumar | A+    |
| 1   | Durga | 102      | Data Structures             | Dr. Ravi  | B+    |
| 3   | Priya | 102      | Data Structures             | Dr. Ravi  | A     |
| 2   | Ravi  | 103      | Machine Learning            | Dr. Priya | A     |
| 4   | Arun  | 103      | Machine Learning            | Dr. Priya | B+    |

## Conclusion

The original ENROLLMENT relation has a composite key (SID, CourseID) and contains partial dependencies. Decomposing it into **STUDENT**, **COURSE**, and **ENROLLMENT** removes these partial dependencies and brings the database into 2NF.

## 4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF

### Airline Booking — BCNF Normalization

Given:

**BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)**

#### 1. Functional Dependencies

Assume the following dependencies:

PassID → PassName

FlightNo → DeptCity, ArrCity

(FlightNo, Date, Seat) → PassID

(FlightNo, Date, PassID) → Seat

Here, a flight can operate on different dates, and each seat on a particular flight/date is assigned to at most one passenger.

#### 2. Candidate Keys

Two possible candidate keys are:

(FlightNo, Date, Seat)

(FlightNo, Date, PassID)

Both can identify a unique booking.

### 3. BCNF Violations

A relation is in **BCNF** if for every FD:

**$X \rightarrow Y$ , X must be a superkey.**

Consider:

**PassID  $\rightarrow$  PassName**

PassID is not a superkey of the complete BOOKING relation.

Also:

**FlightNo  $\rightarrow$  DeptCity, ArrCity**

FlightNo is not a superkey because the same flight may occur on multiple dates.

Therefore, the original relation violates **BCNF**.

### 4. BCNF Decomposition

#### **PASSENGER**

PASSENGER(PassID, PassName)

Primary Key:

**PassID**

FD:

PassID  $\rightarrow$  PassName

Since PassID is the key, this relation is in BCNF.

#### **FLIGHT**

FLIGHT(FlightNo, DeptCity, ArrCity)

Primary Key:

**FlightNo**

FD:

FlightNo  $\rightarrow$  DeptCity, ArrCity

Since FlightNo is the key, this relation is in BCNF.

#### **BOOKING**

BOOKING(FlightNo, Date, PassID, Seat)

Primary Key:

**(FlightNo, Date, PassID)**

and assuming each seat is assigned to only one passenger:

(FlightNo, Date, Seat) → PassID

(FlightNo, Date, PassID) → Seat

Both determinants are candidate keys, so the relation is in BCNF.

**SQL:**

```
1 CREATE TABLE PASSENGER (
2   PassID INT PRIMARY KEY,
3   PassName VARCHAR(50) NOT NULL
4 );
5
6 CREATE TABLE FLIGHT (
7   FlightNo VARCHAR(10) PRIMARY KEY,
8   DeptCity VARCHAR(50) NOT NULL,
9   ArrCity VARCHAR(50) NOT NULL
10 );
11
12 CREATE TABLE BOOKING (
13   FlightNo VARCHAR(10),
14   BookingDate DATE,
15   PassID INT,
16   Seat VARCHAR(10),
17   PRIMARY KEY (FlightNo, BookingDate, PassID),
18   FOREIGN KEY (FlightNo) REFERENCES FLIGHT(FlightNo),
19   FOREIGN KEY (PassID) REFERENCES PASSENGER(PassID),
20   UNIQUE (FlightNo, BookingDate, Seat)
21 );
22
23 INSERT INTO PASSENGER VALUES
24 (101, 'Durga'),
25 (102, 'Ravi'),
26 (103, 'Priya'),
27 (104, 'Arun');
28
29 INSERT INTO FLIGHT VALUES
30 ('AI101', 'Chennai', 'Delhi'),
31 ('AI202', 'Mumbai', 'Bangalore'),
32 ('AI303', 'Hyderabad', 'Chennai');
33
34 INSERT INTO BOOKING VALUES
35 ('AI101', '2026-08-15', 101, '12A'),
36 ('AI101', '2026-08-15', 102, '12B'),
37 ('AI202', '2026-08-16', 103, '15A'),
38 ('AI202', '2026-08-16', 104, '15C'),
39 ('AI303', '2026-08-17', 104, '10C'),
40 ('AI202', '2026-08-16', 101, '15B');
41
42 SELECT ...
```

Output:

| FlightNo | PassID | PassName | Seat | DeptCity  | ArrCity   | BookingDate |
|----------|--------|----------|------|-----------|-----------|-------------|
| AI101    | 101    | Durga    | 12A  | Chennai   | Delhi     | 2026-08-15  |
| AI101    | 102    | Ravi     | 12B  | Chennai   | Delhi     | 2026-08-15  |
| AI202    | 101    | Durga    | 15B  | Mumbai    | Bangalore | 2026-08-16  |
| AI202    | 103    | Priya    | 15A  | Mumbai    | Bangalore | 2026-08-16  |
| AI303    | 104    | Arun     | 10C  | Hyderabad | Chennai   | 2026-08-17  |

| PassID | PassName |
|--------|----------|
| 101    | Durga    |
| 102    | Ravi     |
| 103    | Priya    |
| 104    | Arun     |

| FlightNo | DeptCity  | ArrCity   |
|----------|-----------|-----------|
| AI101    | Chennai   | Delhi     |
| AI202    | Mumbai    | Bangalore |
| AI303    | Hyderabad | Chennai   |

| FlightNo | BookingDate | PassID | Seat |
|----------|-------------|--------|------|
| AI101    | 2026-08-15  | 101    | 12A  |
| AI101    | 2026-08-15  | 102    | 12B  |
| AI202    | 2026-08-16  | 103    | 15A  |
| AI202    | 2026-08-16  | 104    | 15C  |
| AI303    | 2026-08-17  | 104    | 10C  |
| AI202    | 2026-08-16  | 101    | 15B  |

## Final Result

The original BOOKING relation is decomposed into:

**PASSENGER(PassID, PassName)**

**FLIGHT(FlightNo, DeptCity, ArrCity)**

**BOOKING(FlightNo, Date, PassID, Seat)**

This decomposition removes the BCNF violations caused by PassID → PassName and

FlightNo → DeptCity, ArrCity, reducing redundancy and preventing update, insertion, and deletion anomalies.

## 5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF

### HR System – Functional and Transitive Dependencies

Consider the HR relation:

**EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, LocationID, LocationName, Salary)**

Assume the following business rules:

- Each employee has one department.
- Each department has one department name and one manager.
- Each manager is also an employee.
- Each department is located at one location.
- Each location has one location name.
- Employee salary is determined by the employee.

#### 1. Functional Dependencies

The important FDs are:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{Salary}$

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}, \text{LocationID}$

$\text{ManagerID} \rightarrow \text{ManagerName}$

$\text{LocationID} \rightarrow \text{LocationName}$

Therefore, the dependency chain includes:

$\text{EmpID} \rightarrow \text{DeptID}$

$\text{DeptID} \rightarrow \text{DeptName}$

and

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

Also:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

and:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{LocationID} \rightarrow \text{LocationName}$

#### 2. Candidate Key

EmpID uniquely identifies an employee.

Therefore:

**Candidate Key = EmpID**

### 3. Transitive Dependencies

A transitive dependency occurs when:

**$A \rightarrow B$  and  $B \rightarrow C$ , therefore  $A \rightarrow C$**

The transitive dependencies are:

#### **Department**

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

Therefore:

**$\text{EmpID} \rightarrow \text{DeptName}$**  is a transitive dependency.

#### **Manager**

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

Therefore:

**$\text{EmpID} \rightarrow \text{ManagerName}$**  is transitive.

#### **Location**

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{LocationID} \rightarrow \text{LocationName}$

Therefore:

**$\text{EmpID} \rightarrow \text{LocationName}$**  is transitive.

These dependencies cause redundancy in the original HR table.

### 4. Problems in the Original Table

#### **Update Anomaly**

If the name of a department changes, every employee belonging to that department must be updated.

#### **Insertion Anomaly**

A new department cannot be stored unless at least one employee is assigned to it.

#### **Deletion Anomaly**

If the last employee in a department is deleted, department information may also be lost.

#### **Redundancy**

Department, manager, and location information is repeated for every employee.

### 5. 3NF Decomposition

To achieve 3NF, separate attributes based on their determinants.

## **EMPLOYEE**

EMPLOYEE(EmpID, EmpName, DeptID, Salary)

Primary Key:

**EmpID**

FD:

$\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{Salary}$

## **DEPARTMENT**

DEPARTMENT(DeptID, DeptName, ManagerID, LocationID)

Primary Key:

**DeptID**

FD:

$\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}, \text{LocationID}$

## **MANAGER**

MANAGER(ManagerID, ManagerName)

Primary Key:

**ManagerID**

FD:

$\text{ManagerID} \rightarrow \text{ManagerName}$

## **LOCATION**

LOCATION(LocationID, LocationName)

Primary Key:

**LocationID**

FD:

$\text{LocationID} \rightarrow \text{LocationName}$

SQL:

The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```

1 CREATE TABLE MANAGER (
2   ManagerID INT PRIMARY KEY,
3   ManagerName VARCHAR(50) NOT NULL
4 );
5
6 CREATE TABLE LOCATION (
7   LocationID INT PRIMARY KEY,
8   LocationName VARCHAR(100) NOT NULL
9 );
10
11 CREATE TABLE DEPARTMENT (
12   DeptID INT PRIMARY KEY,
13   DeptName VARCHAR(100) NOT NULL,
14   ManagerID INT,
15   LocationID INT,
16   FOREIGN KEY (ManagerID) REFERENCES MANAGER(ManagerID),
17   FOREIGN KEY (LocationID) REFERENCES LOCATION(LocationID)
18 );
19
20 CREATE TABLE EMPLOYEE (
21   EmpID INT PRIMARY KEY,
22   EmpName VARCHAR(50) NOT NULL,
23   DeptID INT,
24   Salary DECIMAL(10,2),
25   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
26 );
27
28 INSERT INTO MANAGER VALUES
29 (501, 'John Smith'),
30 (502, 'Sarah Wilson'),
31 (503, 'David Brown');
32
33 INSERT INTO LOCATION VALUES
34 (1, 'Chennai'),
35 (2, 'Bangalore'),
36 (3, 'Hyderabad');

```

The right pane shows the output of the queries, which is a table with 6 columns: EmpID, EmpName, Salary, DeptID, DeptName, and ManagerID. The data is as follows:

| EmpID | EmpName | Salary   | DeptID | DeptName               | ManagerID |
|-------|---------|----------|--------|------------------------|-----------|
| 1001  | Durga   | 45000.00 | 101    | Human Resources        | 501       |
| 1002  | Ravi    | 48000.00 | 101    | Human Resources        | 501       |
| 1003  | Priya   | 55000.00 | 102    | Finance                | 502       |
| 1004  | Arun    | 60000.00 | 103    | Information Technology | 503       |
| 1005  | Kumar   | 58000.00 | 103    | Information Technology | 503       |

## Conclusion

The original HR relation contains several **functional dependencies** and **transitive dependencies**, particularly:

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{LocationID} \rightarrow \text{LocationName}$

Decomposing it into **EMPLOYEE, DEPARTMENT, MANAGER, and LOCATION** removes the transitive dependencies and redundancy, resulting in a **3NF database design**.

## 6.Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

### Library Relation – 4NF Decomposition

Given:

**LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)**

#### 1. Functional Dependencies

The important FDs are:

$\text{MemberID} \rightarrow \text{MemberName}$

$\text{BookID} \rightarrow \text{Title, Author, Genre}$

$(\text{MemberID, BookID}) \rightarrow \text{BorrowDate}$

#### 2. Multivalued Dependency

A member can borrow multiple books independently of the details of other books.

Therefore, the main MVD is:

$\text{MemberID} \twoheadrightarrow \text{BookID}$

This means one MemberID can be associated with multiple BookID values independently.

For example:

**MemberID MemberName BookID Title**

|     |       |     |        |
|-----|-------|-----|--------|
| M01 | Durga | B01 | DBMS   |
| M01 | Durga | B02 | Python |
| M01 | Durga | B03 | AI     |

The repeated book information creates redundancy.

### 3. Why the Relation Violates 4NF

A relation is in **4NF** if for every non-trivial MVD:

$\mathbf{X} \twoheadrightarrow \mathbf{Y}$

X must be a superkey.

Here:

$\text{MemberID} \twoheadrightarrow \text{BookID}$

But MemberID is **not a superkey** of the original relation.

Therefore, the relation violates **4NF**.

### 4. Decomposition into 4NF

#### **MEMBER**

MEMBER(MemberID, MemberName)

**Primary Key:** MemberID

$\text{MemberID} \rightarrow \text{MemberName}$

#### **BOOK**

BOOK(BookID, Title, Author, Genre)

**Primary Key:** BookID

$\text{BookID} \rightarrow \text{Title, Author, Genre}$

#### **BORROW**

BORROW(MemberID, BookID, BorrowDate)

**Composite Primary Key:** (MemberID, BookID, BorrowDate)

This table records which member borrowed which book and when.

SQL:

The screenshot shows the OneCompiler SQL editor with a MySQL database. The SQL code defines three tables: MEMBER, BOOK, and BORROW. The MEMBER table has columns MemberID (primary key) and MemberName. The BOOK table has columns BookID (primary key), Title, Author, and Genre. The BORROW table has columns MemberID, BookID, and BorrowDate, with foreign keys to MEMBER and BOOK. The code also includes INSERT statements for data. The output window shows the results of the queries, displaying the data for each table in a tabular format.

```
1 CREATE TABLE MEMBER (  
2   MemberID INT PRIMARY KEY,  
3   MemberName VARCHAR(50) NOT NULL  
4 );  
5  
6 CREATE TABLE BOOK (  
7   BookID INT PRIMARY KEY,  
8   Title VARCHAR(100) NOT NULL,  
9   Author VARCHAR(100) NOT NULL,  
10  Genre VARCHAR(50)  
11 );  
12  
13 CREATE TABLE BORROW (  
14   MemberID INT,  
15   BookID INT,  
16   BorrowDate DATE,  
17   PRIMARY KEY (MemberID, BookID, BorrowDate),  
18   FOREIGN KEY (MemberID) REFERENCES MEMBER(MemberID),  
19   FOREIGN KEY (BookID) REFERENCES BOOK(BookID)  
20 );  
21  
22 INSERT INTO MEMBER VALUES  
23 (101, 'Durga'),  
24 (102, 'Ravi'),  
25 (103, 'Priya');  
26  
27 INSERT INTO BOOK VALUES  
28 (201, 'Database Management Systems', 'Korth', 'Database'),  
29 (202, 'Python Programming', 'Mark Lutz', 'Programming'),  
30 (203, 'Artificial Intelligence', 'Russell', 'AI'),  
31 (204, 'Data Structures', 'Seymour Lipschutz', 'Programming');  
32  
33 INSERT INTO BORROW VALUES  
34 (101, 201, '2026-08-01'),  
35 (101, 202, '2026-08-03'),  
36 (101, 203, '2026-08-05'),  
37 (102, 201, '2026-08-02'),  
38 (102, 204, '2026-08-04');
```

Output:

| MemberID | MemberName | BookID | Title                       | Author     |
|----------|------------|--------|-----------------------------|------------|
| 101      | Durga      | 201    | Database Management Systems | Korth      |
| 101      | Durga      | 202    | Python Programming          | Mark Lutz  |
| 101      | Durga      | 203    | Artificial Intelligence     | Russell    |
| 102      | Ravi       | 201    | Database Management Systems | Korth      |
| 102      | Ravi       | 204    | Data Structures             | Seymour Li |
| 103      | Priya      | 203    | Artificial Intelligence     | Russell    |

| MemberID | MemberName |
|----------|------------|
| 101      | Durga      |
| 102      | Ravi       |
| 103      | Priya      |

| BookID | Title                       | Author            | Genre       |
|--------|-----------------------------|-------------------|-------------|
| 201    | Database Management Systems | Korth             | Database    |
| 202    | Python Programming          | Mark Lutz         | Programming |
| 203    | Artificial Intelligence     | Russell           | AI          |
| 204    | Data Structures             | Seymour Lipschutz | Programming |

## Conclusion

The MVD **MemberID**  $\rightarrow\rightarrow$  **BookID** causes the original relation to violate 4NF. Decomposing it into **MEMBER**, **BOOK**, and **BORROW** removes the multivalued dependency and reduces redundancy while preserving the borrowing information.

## 7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

### Telecom Billing Schema – FD and Key Analysis

Consider the following telecom billing relation:

**TELECOM\_BILLING**(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge, CallID, CallDate, CallDuration, CallCharge)

Assume:

- Each customer has one registered phone number.
- Each customer subscribes to one plan.
- Each plan has a fixed name and monthly charge.
- Each call is uniquely identified by CallID.
- A call belongs to a particular customer.

## 1. Functional Dependencies

The main functional dependencies are:

$\text{CustomerID} \rightarrow \text{CustomerName}, \text{PhoneNo}, \text{PlanID}$

$\text{PhoneNo} \rightarrow \text{CustomerID}$

$\text{PlanID} \rightarrow \text{PlanName}, \text{MonthlyCharge}$

$\text{CallID} \rightarrow \text{CustomerID}, \text{CallDate}, \text{CallDuration}, \text{CallCharge}$

Therefore:

$\text{CustomerID} \rightarrow \text{CustomerName}$

$\text{CustomerID} \rightarrow \text{PhoneNo}$

$\text{CustomerID} \rightarrow \text{PlanID}$

$\text{PhoneNo} \rightarrow \text{CustomerID}$

$\text{PlanID} \rightarrow \text{PlanName}$

$\text{PlanID} \rightarrow \text{MonthlyCharge}$

$\text{CallID} \rightarrow \text{CustomerID}$

$\text{CallID} \rightarrow \text{CallDate}$

$\text{CallID} \rightarrow \text{CallDuration}$

$\text{CallID} \rightarrow \text{CallCharge}$

## 2. Candidate Keys

A **candidate key** is a minimal attribute set that uniquely identifies every tuple.

### Candidate Key 1: CallID

Since:

$\text{CallID} \rightarrow \text{CustomerID}, \text{CallDate}, \text{CallDuration}, \text{CallCharge}$

and:

$\text{CustomerID} \rightarrow \text{CustomerName}, \text{PhoneNo}, \text{PlanID}$

$\text{PlanID} \rightarrow \text{PlanName}, \text{MonthlyCharge}$

we can derive all attributes.

Therefore:

**CallID is a candidate key.**

### Candidate Key 2: (CustomerID, CallDate, CallDuration)

If the business rule guarantees that a customer cannot make two calls with the same date and duration, then:

$(\text{CustomerID}, \text{CallDate}, \text{CallDuration}) \rightarrow \text{CallID}$

Under this assumption, this combination can also be a candidate key.

However, **in a practical telecom system, this is usually not a reliable candidate key**, because two calls can have the same date and duration.

Therefore, the safer candidate key is:

**{CallID}**

### **Candidate Key 3: PhoneNo**

Because:

PhoneNo  $\rightarrow$  CustomerID

and:

CustomerID  $\rightarrow$  all customer-related attributes

PhoneNo can uniquely identify the customer, but **it cannot identify individual calls**.

Therefore, for the complete relation, PhoneNo is **not a candidate key**.

### **3. Primary Key**

The most appropriate primary key is:

**Primary Key = CallID**

because every billing/call record has a unique CallID.

### **4. Attribute Classification**

| <b>Attribute</b> | <b>Determined By Type</b> |                    |
|------------------|---------------------------|--------------------|
| CustomerID       | CallID                    | Non-key            |
| CustomerName     | CustomerID                | Non-key            |
| PhoneNo          | CustomerID                | Non-key            |
| PlanID           | CustomerID                | Non-key            |
| PlanName         | PlanID                    | Non-key            |
| MonthlyCharge    | PlanID                    | Non-key            |
| CallID           | —                         | <b>Primary Key</b> |
| CallDate         | CallID                    | Non-key            |
| CallDuration     | CallID                    | Non-key            |
| CallCharge       | CallID                    | Non-key            |

### **5. Transitive Dependencies**

There are several transitive dependencies.

### Customer Details

$\text{CallID} \rightarrow \text{CustomerID} \rightarrow \text{CustomerName}$

Therefore:

**CallID**  $\rightarrow$  **CustomerName** is transitive.

Similarly:

$\text{CallID} \rightarrow \text{CustomerID} \rightarrow \text{PhoneNo}$

$\text{CallID} \rightarrow \text{CustomerID} \rightarrow \text{PlanID}$

### Plan Details

$\text{CallID} \rightarrow \text{CustomerID} \rightarrow \text{PlanID} \rightarrow \text{PlanName}$

Therefore:

**CallID**  $\rightarrow$  **PlanName** is transitive.

Also:

$\text{CallID} \rightarrow \text{CustomerID} \rightarrow \text{PlanID} \rightarrow \text{MonthlyCharge}$

Therefore:

**CallID**  $\rightarrow$  **MonthlyCharge** is transitive.

## 6. Complete FD Set

For the given schema, the important functional dependencies before normalization are:

$\text{CustomerID} \rightarrow \text{CustomerName}, \text{PhoneNo}, \text{PlanID}$

$\text{PhoneNo} \rightarrow \text{CustomerID}$

$\text{PlanID} \rightarrow \text{PlanName}, \text{MonthlyCharge}$

$\text{CallID} \rightarrow \text{CustomerID}, \text{CallDate}, \text{CallDuration}, \text{CallCharge}$

From these, we can derive:

$\text{CallID} \rightarrow \text{CustomerName}$

$\text{CallID} \rightarrow \text{PhoneNo}$

$\text{CallID} \rightarrow \text{PlanID}$

$\text{CallID} \rightarrow \text{PlanName}$

$\text{CallID} \rightarrow \text{MonthlyCharge}$

## 7. Summary

| Item        | Result          |
|-------------|-----------------|
| Relation    | TELECOM_BILLING |
| Primary Key | CallID          |

| Item                         | Result                                                                                        |
|------------------------------|-----------------------------------------------------------------------------------------------|
| Candidate Key                | CallID                                                                                        |
| Alternate candidate key      | None under the stated assumptions                                                             |
| Foreign-key-like attributes  | CustomerID, PlanID                                                                            |
| Main FD                      | CallID $\rightarrow$ CustomerID, CallDate, CallDuration, CallCharge                           |
| Customer FD                  | CustomerID $\rightarrow$ CustomerName, PhoneNo, PlanID                                        |
| Plan FD                      | PlanID $\rightarrow$ PlanName, MonthlyCharge                                                  |
| Additional FD                | PhoneNo $\rightarrow$ CustomerID                                                              |
| Main transitive dependencies | CallID $\rightarrow$ CustomerID $\rightarrow$ PlanID $\rightarrow$ PlanName,<br>MonthlyCharge |

## SQL:

The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```

1 CREATE TABLE TELECOM_BILLING (
2   CustomerID INT,
3   CustomerName VARCHAR(50),
4   PhoneNo VARCHAR(15),
5   PlanID INT,
6   PlanName VARCHAR(50),
7   MonthlyCharge DECIMAL(10,2),
8   CallID INT PRIMARY KEY,
9   CallDate DATE,
10  CallDuration INT,
11  CallCharge DECIMAL(10,2)
12 );
13 INSERT INTO TELECOM_BILLING VALUES
14 (101, 'Durga', '9876543210', 1, 'Basic', 299.00,
15 1001, '2026-08-01', 10, 20.00),
16
17 (101, 'Durga', '9876543210', 1, 'Basic', 299.00,
18 1002, '2026-08-02', 15, 30.00),
19
20 (102, 'Ravi', '9876543211', 2, 'Premium', 599.00,
21 1003, '2026-08-02', 20, 50.00),
22
23 (103, 'Priya', '9876543212', 3, 'Unlimited', 999.00,
24 1004, '2026-08-03', 25, 0.00),
25
26 (102, 'Ravi', '9876543211', 2, 'Premium', 599.00,
27 1005, '2026-08-04', 12, 30.00);
28 SELECT * FROM TELECOM_BILLING;

```

The right pane shows the query results in a table format:

| CustomerID | CustomerName | PhoneNo    | PlanID | PlanName  | MonthlyCharge |
|------------|--------------|------------|--------|-----------|---------------|
| 101        | Durga        | 9876543210 | 1      | Basic     | 299.00        |
| 101        | Durga        | 9876543210 | 1      | Basic     | 299.00        |
| 102        | Ravi         | 9876543211 | 2      | Premium   | 599.00        |
| 103        | Priya        | 9876543212 | 3      | Unlimited | 999.00        |
| 102        | Ravi         | 9876543211 | 2      | Premium   | 599.00        |

## Conclusion:

Before normalization, CallID is the safest candidate and primary key for the complete telecom billing relation. The dependencies CustomerID  $\rightarrow$  CustomerName, PhoneNo, PlanID and PlanID  $\rightarrow$  PlanName, MonthlyCharge create transitive dependencies and indicate that the schema should later be decomposed into separate **CUSTOMER**, **PLAN**, and **CALL/BILLING** relations.

**8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.**

### **Retail Inventory System – Complete Normalization**

**Given relation:**

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

**Assume:**

- Each PID identifies one product.
- Each product has one supplier.
- Each SupplierID identifies one supplier.
- Each product belongs to one category.
- Each product is stored in one warehouse.

#### **1. Functional Dependencies**

The main FDs are:

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

$SupplierID \rightarrow SupplierName$

Therefore, there is a transitive dependency:

$PID \rightarrow SupplierID \rightarrow SupplierName$

The candidate key is:

PID

#### **2. First Normal Form (1NF)**

The relation already satisfies 1NF because all attributes contain atomic values.

PRODUCT(

PID,

PName,

SupplierID,

SupplierName,

Category,

Price,

Warehouse

)

### 3. Second Normal Form (2NF)

The primary key is a single attribute PID.

Therefore, there cannot be any partial dependency on a part of the key.

So, the relation is already in 2NF.

### 4. Third Normal Form (3NF)

There is a transitive dependency:

$PID \rightarrow SupplierID$

$SupplierID \rightarrow SupplierName$

Therefore:

$PID \rightarrow SupplierName$

is a transitive dependency.

To remove it, decompose the relation.

PRODUCT

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

Primary Key: PID

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

SUPPLIER

SUPPLIER(SupplierID, SupplierName)

Primary Key: SupplierID

$SupplierID \rightarrow SupplierName$

### 5. BCNF Check

For PRODUCT:

$PID \rightarrow PName, SupplierID, Category, Price, Warehouse$

PID is the candidate key, so the determinant is a superkey.

For SUPPLIER:

$SupplierID \rightarrow SupplierName$

SupplierID is the candidate key, so the determinant is a superkey.

Therefore, both relations satisfy BCNF under the stated assumptions.

Final Normalized Schema

SUPPLIER

SupplierID    PRIMARY KEY  
SupplierName

PRODUCT

PID            PRIMARY KEY  
PName  
SupplierID    FOREIGN KEY  
Category  
Price  
Warehouse

Normalization Summary

Normal Form Result

- 1NF            Satisfied – atomic attributes
- 2NF            Satisfied – single-attribute key
- 3NF            Decompose due to PID → SupplierID → SupplierName
- BCNF          Satisfied after decomposition

SQL:

OneCompiler

MySQL

Run

queries.sql

```
1 CREATE TABLE SUPPLIER (  
2   SupplierID INT PRIMARY KEY,  
3   SupplierName VARCHAR(100) NOT NULL  
4 );  
5 CREATE TABLE PRODUCT (  
6   PID INT PRIMARY KEY,  
7   PName VARCHAR(100) NOT NULL,  
8   SupplierID INT,  
9   Category VARCHAR(50),  
10  Price DECIMAL(10,2),  
11  Warehouse VARCHAR(100),  
12  FOREIGN KEY (SupplierID) REFERENCES SUPPLIER(SupplierID)  
13 );  
14 INSERT INTO SUPPLIER VALUES  
15 (101, 'ABC Suppliers'),  
16 (102, 'XYZ Distributors'),  
17 (103, 'Global Traders'),  
18 (104, 'Tech Supplies');  
19 INSERT INTO PRODUCT VALUES  
20 (1, 'Laptop', 101, 'Electronics', 55000.00, 'Chennai Warehouse'),  
21 (2, 'Mouse', 102, 'Accessories', 800.00, 'Chennai Warehouse'),  
22 (3, 'Keyboard', 102, 'Accessories', 1500.00, 'Bangalore Warehouse'),  
23 (4, 'Monitor', 103, 'Electronics', 12000.00, 'Hyderabad Warehouse'),  
24 (5, 'Printer', 104, 'Office Equipment', 15000.00, 'Bangalore Warehouse');  
25 SELECT * FROM SUPPLIER;  
26 SELECT * FROM PRODUCT;
```

Output:

| SupplierID | SupplierName     |
|------------|------------------|
| 101        | ABC Suppliers    |
| 102        | XYZ Distributors |
| 103        | Global Traders   |
| 104        | Tech Supplies    |

| PID | PName    | SupplierID | Category         | Price    | Warehouse       |
|-----|----------|------------|------------------|----------|-----------------|
| 1   | Laptop   | 101        | Electronics      | 55000.00 | Chennai Warehou |
| 2   | Mouse    | 102        | Accessories      | 800.00   | Chennai Warehou |
| 3   | Keyboard | 102        | Accessories      | 1500.00  | Bangalore Ware  |
| 4   | Monitor  | 103        | Electronics      | 12000.00 | Hyderabad Ware  |
| 5   | Printer  | 104        | Office Equipment | 15000.00 | Bangalore Ware  |

**Final result:**

The original PRODUCT relation is completely normalized into **PRODUCT** and **SUPPLIER**, eliminating the transitive dependency and reducing update, insertion, and deletion anomalies.

**9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification****School ERP – Complete Normalization up to BCNF****Consider a poorly normalized School ERP relation:**

SCHOOL\_ERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Marks, ExamID, ExamName)

Assume:

- Each StudentID identifies one student.
- Each ClassID identifies one class.
- Each TeacherID identifies one teacher.
- Each SubjectID identifies one subject.
- Each ExamID identifies one exam.
- A student can study multiple subjects.
- A subject can be taught by different teachers/classes.
- Marks are recorded for a student in a particular subject and exam.

**1. Functional Dependencies**

The important functional dependencies are:

StudentID  $\rightarrow$  StudentName, ClassID

ClassID  $\rightarrow$  ClassName

TeacherID  $\rightarrow$  TeacherName

SubjectID  $\rightarrow$  SubjectName

ExamID  $\rightarrow$  ExamName

(StudentID, SubjectID, ExamID)  $\rightarrow$  Marks

There are also transitive dependencies:

StudentID  $\rightarrow$  ClassID  $\rightarrow$  ClassName

Therefore:

StudentID  $\rightarrow$  ClassName

is a transitive dependency.

## 2. Candidate Key

For a student's marks, we need to identify:

- Which student?
- Which subject?
- Which exam?

Therefore:

Candidate Key = (StudentID, SubjectID, ExamID)

The composite key determines Marks.

(StudentID, SubjectID, ExamID) → Marks

## 3. Problems in the Poorly Normalized Schema

Update Anomaly

If a teacher's name changes, multiple records may need to be updated.

Insertion Anomaly

A new subject cannot be stored unless it is associated with a student/exam record.

Deletion Anomaly

Deleting the last student record for a subject may also remove the subject's information.

Redundancy

Names of students, classes, teachers, subjects, and exams are repeated in many records.

## 4. 1NF

The original relation can be considered in 1NF if each field contains a single atomic value.

SCHOOL\_ERP(

StudentID,

StudentName,

ClassID,

ClassName,

TeacherID,

TeacherName,

SubjectID,

SubjectName,

Marks,

ExamID,

ExamName

)

## 5. 2NF

The candidate key is:

(StudentID, SubjectID, ExamID)

But several attributes depend only on part of the composite key:

StudentID  $\rightarrow$  StudentName, ClassID

SubjectID  $\rightarrow$  SubjectName

ExamID  $\rightarrow$  ExamName

These are partial dependencies, so the relation is not in 2NF.

Decompose into separate entities.

## 6. 3NF

There are transitive dependencies such as:

StudentID  $\rightarrow$  ClassID  $\rightarrow$  ClassName

Therefore, ClassName should be separated from the student table.

Similarly, each entity's descriptive information should be stored separately.

## 7. BCNF Decomposition

STUDENT

STUDENT(  
    StudentID,  
    StudentName,  
    ClassID  
)

Primary Key: StudentID

FD:

StudentID  $\rightarrow$  StudentName, ClassID

CLASS

CLASS(  
    ClassID,  
    ClassName  
)

Primary Key: ClassID

FD:

ClassID  $\rightarrow$  ClassName

TEACHER

TEACHER(  
    TeacherID,  
    TeacherName

)

Primary Key: TeacherID

FD:

TeacherID  $\rightarrow$  TeacherName

SUBJECT

SUBJECT(  
    SubjectID,  
    SubjectName

)

Primary Key: SubjectID

FD:

SubjectID  $\rightarrow$  SubjectName

EXAM

EXAM(  
    ExamID,  
    ExamName

)

Primary Key: ExamID

FD:

ExamID  $\rightarrow$  ExamName

TEACHING

To represent which teacher teaches which subject in a class:

TEACHING(  
    TeacherID,  
    SubjectID,  
    ClassID

)

Composite Primary Key:

(TeacherID, SubjectID, ClassID)

This avoids repeating teacher and subject information.

MARKS

MARKS(

StudentID,

SubjectID,

ExamID,

Marks

)

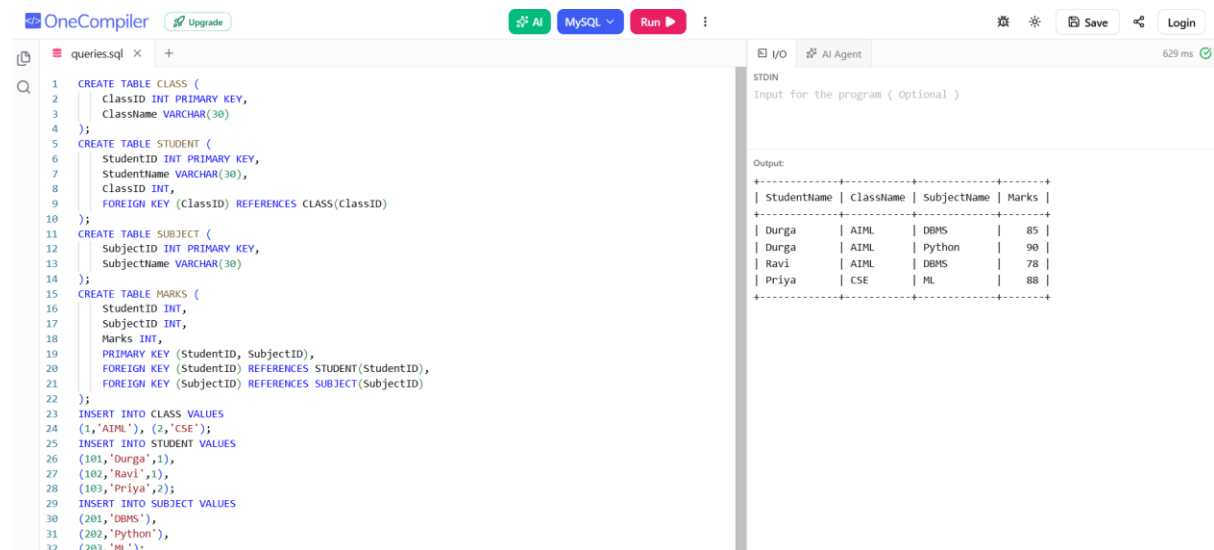
Composite Primary Key:

(StudentID, SubjectID, ExamID)

FD:

(StudentID, SubjectID, ExamID) → Marks

SQL:



The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```
1 CREATE TABLE CLASS (  
2   ClassID INT PRIMARY KEY,  
3   ClassName VARCHAR(30)  
4 );  
5 CREATE TABLE STUDENT (  
6   StudentID INT PRIMARY KEY,  
7   StudentName VARCHAR(30),  
8   ClassID INT,  
9   FOREIGN KEY (ClassID) REFERENCES CLASS(ClassID)  
10 );  
11 CREATE TABLE SUBJECT (  
12   SubjectID INT PRIMARY KEY,  
13   SubjectName VARCHAR(30)  
14 );  
15 CREATE TABLE MARKS (  
16   StudentID INT,  
17   SubjectID INT,  
18   Marks INT,  
19   PRIMARY KEY (StudentID, SubjectID),  
20   FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID),  
21   FOREIGN KEY (SubjectID) REFERENCES SUBJECT(SubjectID)  
22 );  
23 INSERT INTO CLASS VALUES  
24 (1,'AIML'), (2,'CSE');  
25 INSERT INTO STUDENT VALUES  
26 (101,'Durga',1),  
27 (102,'Ravi',1),  
28 (103,'Priya',2);  
29 INSERT INTO SUBJECT VALUES  
30 (201,'DBMS'),  
31 (202,'Python'),  
32 (203,'ML');
```

The right pane shows the output of the SQL execution, displaying a table with the following data:

| StudentName | Classname | SubjectName | Marks |
|-------------|-----------|-------------|-------|
| Durga       | AIML      | DBMS        | 85    |
| Durga       | AIML      | Python      | 90    |
| Ravi        | AIML      | DBMS        | 78    |
| Priya       | CSE       | ML          | 88    |

## Final Conclusion

The poorly normalized School ERP schema contains **partial dependencies and transitive dependencies**, causing insertion, deletion, update anomalies, and redundancy. Decomposing it into **STUDENT, CLASS, TEACHER, SUBJECT, EXAM, TEACHING, and MARKS** removes these dependencies. Under the stated business rules, each resulting relation satisfies **BCNF**, giving a consistent and scalable School ERP database design.

## 10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification

### Movie Streaming Platform – 5NF Decomposition

Consider the poorly normalized relation:

**STREAMING**(UserID, MovieID, ActorID, UserName, MovieTitle, ActorName)

Assume:

- A user can watch many movies.
- A movie can have many actors.
- An actor can appear in many movies.
- The user-movie relationship and movie-actor relationship are independent.

### 1. Join Dependencies

The important relationships are:

**USER**(UserID, UserName)

**MOVIE**(MovieID, MovieTitle)

**ACTOR**(ActorID, ActorName)

The original relation contains combinations of users, movies, and actors.

A join dependency can be represented as:

**STREAMING** =

**USER\_MOVIE** ⋈ **MOVIE\_ACTOR** ⋈ **USER\_ACTOR**

where:

**USER\_MOVIE**(UserID, MovieID)

**MOVIE\_ACTOR**(MovieID, ActorID)

**USER\_ACTOR**(UserID, ActorID)

So the join dependency is:

$\bowtie\{(UserID, MovieID), (MovieID, ActorID), (UserID, ActorID)\}$

### 2. Why 5NF is Required

A relation is in **5NF (Project-Join Normal Form)** when every non-trivial join dependency is implied by the candidate keys.

Suppose the original relation contains:

**UserID MovieID ActorID**

U01     M01     A01

**UserID MovieID ActorID**

U01     M01     A02

U02     M01     A01

Storing all three relationships together can create unnecessary combinations and redundancy. Therefore, we decompose the relation into smaller relations based on the independent relationships.

### 3. 5NF Decomposition

**USER\_MOVIE**

USER\_MOVIE(UserID, MovieID)

**Primary Key:** (UserID, MovieID)

This records which movies a user watches.

**MOVIE\_ACTOR**

MOVIE\_ACTOR(MovieID, ActorID)

**Primary Key:** (MovieID, ActorID)

This records which actors appear in which movies.

**USER\_ACTOR**

USER\_ACTOR(UserID, ActorID)

**Primary Key:** (UserID, ActorID)

This records the user-actor relationship where such a relationship is independently required.

### 4. Lossless Join Verification

The original relation can be reconstructed using:

USER\_MOVIE

⋈

MOVIE\_ACTOR

⋈

USER\_ACTOR

In relational algebra:

STREAMING

=

USER\_MOVIE

⋈

MOVIE\_ACTOR

⋈

USER\_ACTOR

If joining the decomposed relations produces exactly the original valid tuples, the decomposition is **lossless**.

### Example

**USER\_MOVIE**

**UserID MovieID**

U01 M01

U02 M01

**MOVIE\_ACTOR**

**MovieID ActorID**

M01 A01

M01 A02

**USER\_ACTOR**

**UserID ActorID**

U01 A01

U01 A02

U02 A01

Joining these relations gives:

**UserID MovieID ActorID**

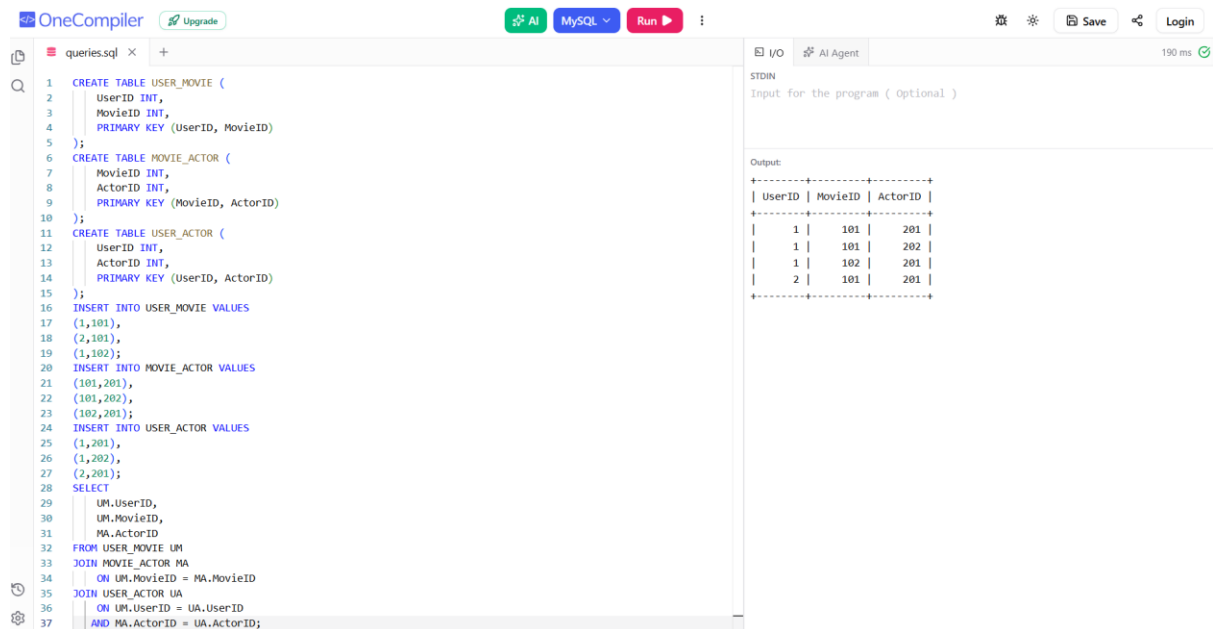
U01 M01 A01

U01 M01 A02

U02 M01 A01

which matches the original valid combinations.

SQL:



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script with the following content:

```
1 CREATE TABLE USER_MOVIE (  
2     UserID INT,  
3     MovieID INT,  
4     PRIMARY KEY (UserID, MovieID)  
5 );  
6 CREATE TABLE MOVIE_ACTOR (  
7     MovieID INT,  
8     ActorID INT,  
9     PRIMARY KEY (MovieID, ActorID)  
10 );  
11 CREATE TABLE USER_ACTOR (  
12     UserID INT,  
13     ActorID INT,  
14     PRIMARY KEY (UserID, ActorID)  
15 );  
16 INSERT INTO USER_MOVIE VALUES  
17 (1,101),  
18 (2,101),  
19 (1,102);  
20 INSERT INTO MOVIE_ACTOR VALUES  
21 (101,201),  
22 (101,202),  
23 (102,201);  
24 INSERT INTO USER_ACTOR VALUES  
25 (1,201),  
26 (1,202),  
27 (2,201);  
28 SELECT  
29     UM.UserID,  
30     UM.MovieID,  
31     MA.ActorID  
32 FROM USER_MOVIE UM  
33 JOIN MOVIE_ACTOR MA  
34     ON UM.MovieID = MA.MovieID  
35 JOIN USER_ACTOR UA  
36     ON UM.UserID = UA.UserID  
37     AND MA.ActorID = UA.ActorID;
```

The right pane shows the output of the query, which is a table with 3 columns: UserID, MovieID, and ActorID. The output is as follows:

| UserID | MovieID | ActorID |
|--------|---------|---------|
| 1      | 101     | 201     |
| 1      | 101     | 202     |
| 1      | 102     | 201     |
| 2      | 101     | 201     |

## Conclusion

The original movie-streaming relation contains a **non-trivial join dependency** among user-movie, movie-actor, and user-actor relationships. Decomposing it into **USER\_MOVIE**, **MOVIE\_ACTOR**, and **USER\_ACTOR** removes the join dependency redundancy and provides a **5NF decomposition**. The original relation can be reconstructed through natural joins, giving a **lossless-join decomposition**.